```python
#!/usr/bin/env python3 -B
"""hw1a.py: accuracy under class imbalance"""
import random
from stats import Confuse, confuse, confused

random.seed(1)

RATIOS = [              # (num_pos, num_neg)
  (50,  50),
  (10,  90),
  (5,   95),
  (1,   99),
  (1,   999)]

TP_RATE = 0.70        # classifier catches 70% of +
FP_RATE = 0.05         # classifier false-alarms 5%

print(f"{'ratio':>10} {'acc':>5} {'pd':>5}"
      f" {'pf':>5} {'prec':>5}")
print("-" * 40)

for n_pos, n_neg in RATIOS:
 cf = Confuse()
 for _ in range(n_pos):
 got = "pos" if _ < int(n_pos * TP_RATE) else "neg"
   confuse(cf, "pos", got)
 for _ in range(n_neg):
 got = "pos" if _ < int(n_neg * FP_RATE) else "neg"
   confuse(cf, "neg", got)
 summary = confused(cf, summary=True)
 ratio_str = f"{n_pos}:{n_neg}"
 print(f"{ratio_str:>8} {summary.acc:>5} {summary.pd:>5} {summary.pf:>5} {summary.prec}")
```

```python
#!/usr/bin/env python3 -B
"""hw1b.py: top() with pragmatic eps"""
import random, math, statistics
from stats import top

random.seed(1)
```

```python
def weibull(n=20):
 shape = random.uniform(0.5, 3)
 scale = random.uniform(1, 4)
 return [min(10, scale*(-math.log(random.random()))**(1/shape)*2.5)
   for _ in range(n)]
sizes = []
for trial in range(50):
 rxs = {i: weibull() for i in range(20)}
 pooled = [x for vals in rxs.values() for x in vals]
 sd     = statistics.stdev(pooled)
 winners = top(rxs, eps=0.35 * sd)
 means = {k: sum(rxs[k]) / len(rxs[k]) for k in winners}
 best = max(means.values())
 winners = {k for k in winners if best - means[k] <= 0.35 * sd}
 sizes.append(len(winners))
 means = [sum(rxs[k]) / len(rxs[k]) for k in winners]

 if len(means) > 1:
   assert max(means) - min(means) <= 0.35 * sd, means

print(f"avg winners: {sum(sizes)/len(sizes):.1f}/20")
print(f"min winners: {min(sizes)}")
print(f"max winners: {max(sizes)}")
```

```python
#!/usr/bin/env python3 -B
"""hw2.py: tournament across MOOT data sets"""
import random, glob, statistics, traceback
from ez import csv, Data, shuffle, main, filename
from sa import sa
from locals import ls, lsRminus, saRplus
from stats import top

ALGOS   = [sa, ls, lsRminus, saRplus]
REPEATS = 20
SAMPLE  = 50

def eg__tour(d:filename):
    "run tournament on moot/optimize/*/*.csv"
    files = glob.glob(d + "/*/*.csv")
    print(f"found {len(files)} csv files")
    wins = {a.__name__: 0 for a in ALGOS}

    for f in sorted(files):
        try:
            d0 = Data(csv(f))
            if len(d0.rows) < SAMPLE: continue
        except Exception:
            continue

        # --- baseline: untreated energies for eps ---
        baseline = []
        for _ in range(REPEATS):
            rows = shuffle(d0.rows[:])[:SAMPLE]
            d1   = Data([d0.cols.names] + rows)
            for r in d1.rows:
                baseline.append(d1.disty(r))
        sd = statistics.stdev(baseline) if len(baseline) > 1 else 1

        seen = {a.__name__: [] for a in ALGOS}
        for _ in range(REPEATS):
            rows = shuffle(d0.rows[:])[:SAMPLE]
            d1   = Data([d0.cols.names] + rows)
            for algo in ALGOS:
                final_e = next(reversed(list(algo(d1))))[1]  # final energy
                seen[algo.__name__].append(int(100 * final_e))

        winners = top(seen, eps=0.35 * sd)
        for w in winners:
            wins[w] += 1

    print(f"\n{'algo':>12} {'wins':>6}")
    print("-" * 25)
    for name in sorted(wins):
        print(f"{name:>12} {wins[name]:>6}")

if __name__ == "__main__": main(globals())
```

```python
#!/usr/bin/env python3 -B
"""hw3.py: Welford Num vs reservoir Num"""
import random, math, glob, statistics
from ez import csv, Data, shuffle, Num, col, filename
from sa import sa
from locals import ls, lsRminus, saRplus
from stats import top
import ez
class WelfordNum(Num):
 """Welford online mean/variance. No reservoir.
 Approximate lo/hi as mu +/- 3*sd."""
 def __init__(self, mx=None):
   list.__init__(self)       # empty list (unused)
   self.n    = 0
   self.mu   = 0.0
   self.m2   = 0.0
   self.seen = 0
   self.mx   = mx or 256
 def add(self, v):
   if v == "?": return v
   self.seen += 1
   self.n   += 1
   # Welford incremental update
   delta   = v - self.mu
   self.mu += delta / self.n
   delta2  = v - self.mu
   self.m2 += delta * delta2
   return v
 def sub(self, v):
   if v == "?": return v
   self.seen -= 1
```

```
    if self.n <= 1:
      self.n, self.mu, self.m2 = 0, 0.0, 0.0
      return v
    # reverse Welford
    delta   = v - self.mu
    self.n -= 1
    self.mu -= delta / self.n
    delta2  = v - self.mu
    self.m2 -= delta * delta2
    return v
  def mid(self):
    return self.mu
  def spread(self):
    return math.sqrt(self.m2/(self.n-1)) if self.n>1 else 0
def _lo(self): return self.mu - 3*self.spread()
  def _hi(self): return self.mu + 3*self.spread()
  def norm(self, v):
    if v == "?": return v
    lo, hi = self._lo(), self._hi()
    return 0 if lo==hi else max(0,min(1, (v - lo)/(hi - lo)))
  def pick(self, v=None):
    # gaussian perturbation around v or mu
    base = self.mu if v is None or v=="?" else v
    result = random.gauss(base, self.spread())
    result = max(self._lo(), min(result, self._hi()))
    return result
  def distx(self, u, v):
    if u == v == "?": return 1
    u, v = self.norm(u), self.norm(v)
    u = u if u != "?" else (0 if v > 0.5 else 1)
    v = v if v != "?" else (0 if u > 0.5 else 1)
    return abs(u - v)
  def like(self, v, prior=0):
    s = self.spread() + 1e-32
    return ((1/math.sqrt(2*math.pi*s*s))
            * math.exp(-((v-self.mu)**2)/(2*s*s)))
# --- monkey-patch ---
_original_col = ez.col
def welford_col(s):
 if s[0].isupper(): return WelfordNum()
 return _original_col(s)
def run_tour(files, use_welford=False):
 ez.col = welford_col if use_welford else _original_col
 ALGOS   = [sa, ls, lsRminus, saRplus]
 REPEATS = 20; SAMPLE = 50
 wins = {a.__name__: 0 for a in ALGOS}
 for f in sorted(files):
   try:
     d0 = Data(csv(f))
     if len(d0.rows) < SAMPLE: continue
   except Exception:
     continue
   baseline = []
   for _ in range(REPEATS):
     rows = shuffle(d0.rows[:])[:SAMPLE]
     d1   = Data([d0.cols.names] + rows)
     for r in d1.rows:
       baseline.append(d1.disty(r))
   sd = statistics.stdev(baseline) \
        if len(baseline) > 1 else 1
   seen = {a.__name__: [] for a in ALGOS}
   for _ in range(REPEATS):
     rows = shuffle(d0.rows[:])[:SAMPLE]
     d1 = Data([d0.cols.names] + rows)
     for algo in ALGOS:
       print(f"Running {algo.__name__} on {f}")
       for h, e, row in algo(d1):
           pass
       seen[algo.__name__].append(int(100*e))
   winners = top(seen, eps=0.35 * sd)
   for w in winners: wins[w] += 1
 for name in sorted(wins):
   print(f"  {name:>12} {wins[name]:>6}")
def eg__compare(d:str): #changed filename-> string because you can't pass a directory in as a file name w/o errors
 "parametric vs non-parametric tournament"
 files = glob.glob(d + "/*/*.csv")
 print(f"found {len(files)} csv files")
 print("\n=== RESERVOIR (non-parametric) ===")
 run_tour(files, use_welford=False)
 print("\n=== WELFORD (Gaussian) ===")
 run_tour(files, use_welford=True)
if __name__ == "__main__":
 from ez import main
 main(globals())
```

```
#!/usr/bin/env python3 -B
"""hw4.py: hyperparameter sensitivity (grad)"""
import random, glob, statistics
from ez import csv, Data, shuffle, main, filename
from sa import sa, oneplus1
from locals import ls
from stats import top

SA_MS    = [0.1, 0.3, 0.5, 0.7, 0.9]
LS_RS    = [0, 25, 50, 100, 200]
REPEATS  = 20
SAMPLE   = 50

def eg__hparam(d:filename):
 "hyperparameter sensitivity across MOOT"
 files = glob.glob(d + "/*/*.csv")
 print(f"found {len(files)} csv files")

 # treatment names are tuples
 treatments = ([("sa",m)  for m in SA_MS] +
               [("ls",rs) for rs in LS_RS])
 wins = {t: 0 for t in treatments}

 for f in sorted(files):
   try:
     d0 = Data(csv(f))
     if len(d0.rows) < SAMPLE: continue
   except Exception:
     continue

   # --- baseline eps ---
   baseline = []
   for _ in range(REPEATS):
     rows = shuffle(d0.rows[:])[:SAMPLE]
     d1   = Data([d0.cols.names] + rows)
     for r in d1.rows:
       baseline.append(d1.disty(r))
   sd = statistics.stdev(baseline) \
        if len(baseline) > 1 else 1

   seen = {t: [] for t in treatments}
   for _ in range(REPEATS):
     rows = shuffle(d0.rows[:])[:SAMPLE]
     d1   = Data([d0.cols.names] + rows)
     for name, param in treatments:
       if name == "sa":
         for _, e, _ in sa(d1, m=param): pass  # e gets the last value
       elif name == "ls":
         for _, e, _ in ls(d1, restarts=param): pass
       seen[(name, param)].append(int(100 * e))

   winners = top(seen, eps=0.35 * sd)
   for w in winners: wins[w] += 1

 print(f"\n{'treatment':>15} {'wins':>6}")
 print("-" * 25)
 for t in sorted(wins, key=lambda t: -wins[t]):
   print(f"{str(t):>15} {wins[t]:>6}")

if __name__ == "__main__": eg__hparam("../../moot/optimize")
```

```
#!/usr/bin/env python3 -B
"""hw5.py: sample size sensitivity (grad only)"""
import random, glob, statistics
from ez import csv, Data, shuffle, main, filename
from sa import sa
```

```
from locals import ls, lsRminus, saRplus
from stats import top

ALGOS   = [sa, ls, lsRminus, saRplus]
SAMPLES = [30, 50, 100, 200]
REPEATS = 20

def eg__sample(d:filename):
 "sample size sensitivity across MOOT"
 files = glob.glob(d + "/*/*.csv")
 print(f"found {len(files)} csv files")

 # treatments: (algo_name, sample_size)
 treatments = [(a.__name__, n)
               for a in ALGOS for n in SAMPLES]
 wins = {t: 0 for t in treatments}
 algo_by_name = {a.__name__: a for a in ALGOS}

 for f in sorted(files):
   try:
     d0 = Data(csv(f))
     if len(d0.rows) < max(SAMPLES): continue
   except Exception:
     continue

   # --- baseline eps from largest sample ---
   baseline = []
   for _ in range(REPEATS):
     rows = shuffle(d0.rows[:])[:max(SAMPLES)]
     d1   = Data([d0.cols.names] + rows)
     for r in d1.rows:
       baseline.append(d1.disty(r))
   sd = statistics.stdev(baseline) \
        if len(baseline) > 1 else 1

   seen = {t: [] for t in treatments}
   for _ in range(REPEATS):
     for name, n in treatments:
       rows = shuffle(d0.rows[:])[:n]
       d1   = Data([d0.cols.names] + rows)
       algo = algo_by_name[name]
       for h, e, row in algo(d1): pass
       seen[(name, n)].append(int(100*e))

   winners = top(seen, eps=0.35 * sd)
   # tiebreak: prefer smallest sample
   best_n  = min(n for (_,n) in winners)
   winners = {w for w in winners if w[1]==best_n}
   for w in winners: wins[w] += 1

 print(f"\n{'treatment':>20} {'wins':>6}")
 print("-" * 30)
 for t in sorted(wins, key=lambda t: -wins[t]):
   print(f"{str(t):>20} {wins[t]:>6}")

if __name__ == "__main__":
eg__sample("../../moot/optimize") #main(globals())
```

HW1a

```
# In instances where the positive class is less frequent (most examples are negative) the accuracy will seem like it is really high even if it predicts all true instances incorrectly, because they are so rare. When we see the ratio increasing, as in the cases of true cases becoming more rare, the accuracy is very high. Someone who does not know about this misleading factor of accuracy would assume that it is really good at predictions when in reality, it is not.

     /* ratio   acc    pd    pf  prec */
/* ---------------------------------------- */
   /* 50:50    83    83    17 83 */
   /* 10:90    93    93     7 93 */
    /* 5:95    94    94     6 94 */
```

HW1b

```
# The larger the eps, the more winners there are. Essentially, the larger the value is for eps, there is an increased variance and acceptable range which increases the number of acceptable winners.

/* avg winners: 3.8/20 */
/* min winners: 1 */
/* max winners: 8 */
```

HW2

```
# Restarts typically help algorithms because they allow the searches to escape local optima and prevent them fro     m getting stucl. Because of this, this often increases the chances that there will be more wins. However, there     are instances like small samples or easy landscapes, where restarts don't generally outperform algortihms without restarts.
     /* algo    wins */
/* ------------------------ */
       /*   ls    127 */
    /* lsRminus     16 */
       /*   sa    110 */
     /* saRplus    127 */
```

HW3

```
# Overall, the winners for both reservoir and welford was ls and saRplus. When comparing parametric vs not, there is only minor differences such as 110 vs 123 for sa or 16 vs 22 for lsRminus. In both of these instances, there is improvment, but overall the algorithm seems to be more important.

/* === RESERVOIR (non-parametric) === */
/* ls    127 */
/* lsRminus     16 */
/* sa    110 */
/* saRplus    127 */

/* === WELFORD (Gaussian) === */
/* ls    127 */
/* lsRminus     22 */
/* sa    124 */
/* saRplus    127 */
```

HW4

```
# Based on the table, these seems to be a slight improvement when adjusting the hyperparam     ters. Overall, while tuning can offer some improvement, there should be a biggest emphasis     on algorithm choice. This shows that while improvement can be nice, we should focus more     on improving the algorithm rather than exhaustive tuning.

/* treatment   wins */
/* ------------------------ */
/* ('ls', 25)    127 */
/* ('ls', 50)    127 */
/* ('ls', 100)    125 */
/* ('sa', 0.5)    110 */
/* ('ls', 200)    108 */
/* ('sa', 0.7)    106 */
/* ('sa', 0.9)    106 */
/* ('sa', 0.3)    103 */
/* ('sa', 0.1)    102 */
/* ('ls', 0)     19 */
```

HW5

```
# Interestingly, the highest wins are most often from smaller sample sizes and in this case the sample size is 30. The larger sample sizes do win as well, but there is a pretty steap drop off from 50, 49, 45 to 20, 15, and 13. So, based on this table, and what we have learned in class, less data often wins and is less coss intensive compared to the larger data sets.

/* treatment   wins */
/* ------------------------------ */
/* ('ls', 30)     50 */
/* ('saRplus', 30)     49 */
/* ('sa', 30)     45 */
/* ('saRplus', 200)     20 */
/* ('ls', 200)     15 */
/* ('lsRminus', 30)     15 */
/* ('sa', 200)     13 */
/* ('ls', 100)     11 */
/* ('saRplus', 50)      9 */
/* ('saRplus', 100)      9 */
/* ('sa', 100)      8 */
/* ('sa', 50)      6 */
/* ('ls', 50)      6 */
/* ('lsRminus', 50)      0 */
/* ('lsRminus', 100)      0 */
/* ('lsRminus', 200)      0 */
```